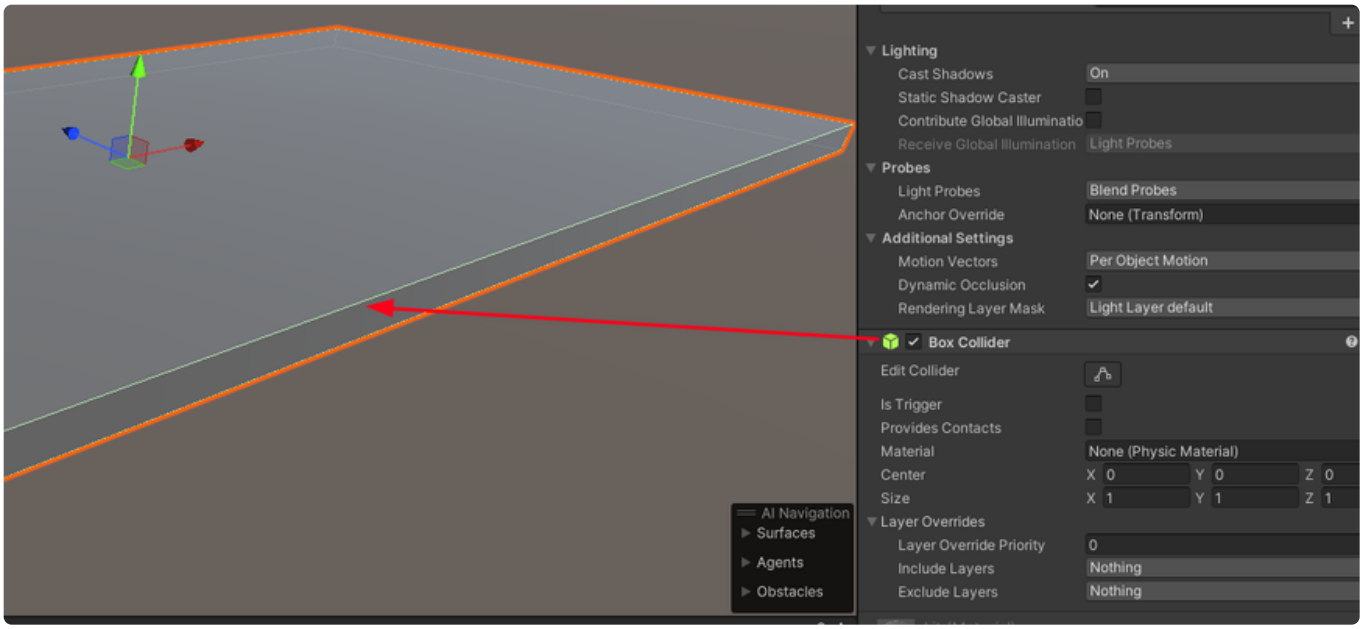
Custom Scenes for H

Refer to this document for extra set-up needed for custom scenes to function in H correctly.

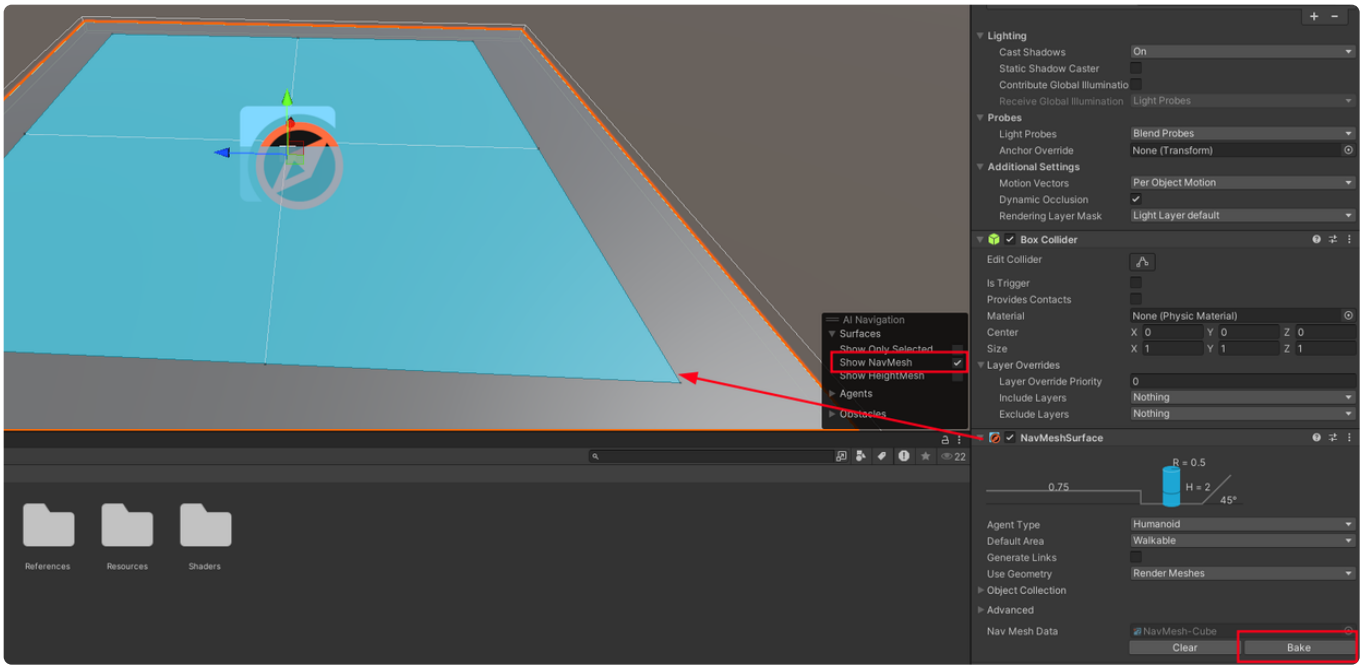
Navigation

During H, both the player and the partner can navigate the map, therefore **Colliders** and a **NavMeshSurface** is needed.

Make sure to place colliders appropriately to prevent the player from falling out of the map or reaching locations not intended to be reached. If you intend to have a lot of colliders, you can simplify collisions and improve performance by using basic shape colliders instead of complex mesh colliders.



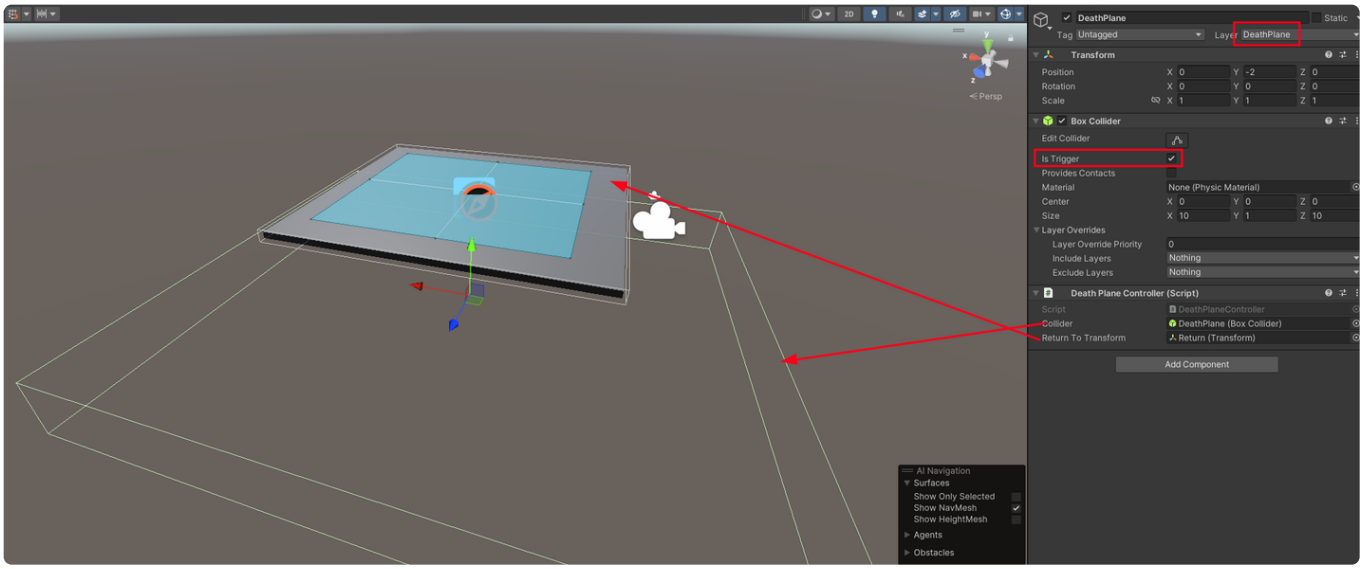
NPCs use a navigation mesh to walk around and therefore need one to exist in the scene. Assign a **NavMeshSurface** component and click the **Bake** button to bake the mesh. Make adjustments as needed.



In cases where you do not want some objects to be part of the navmesh, add a **NavMesh Modifier** component on it and set it up to remove that object from the navmesh. Make sure to re-bake to apply the changes.

To handle rare cases where the player might still fall out of the map, you should create a deathplane GameObject placed under the map that has the **DeathPlaneController** component and an appropriately sized large **Box Collider** component marked as a trigger. Make sure the object containing the collider is set to the **DeathPlane** layer.

Create another GameObject which is where the player will be teleported to in-case the deathplane is reached and assign it alongside the collider to the controller.



Interactions

Parts of the scene can be interactable, for example doors and light switches.

Scene Transition

This interaction is used to transition the player, following partners and any relevant data from one scene to another by creating, merging and loading save files.

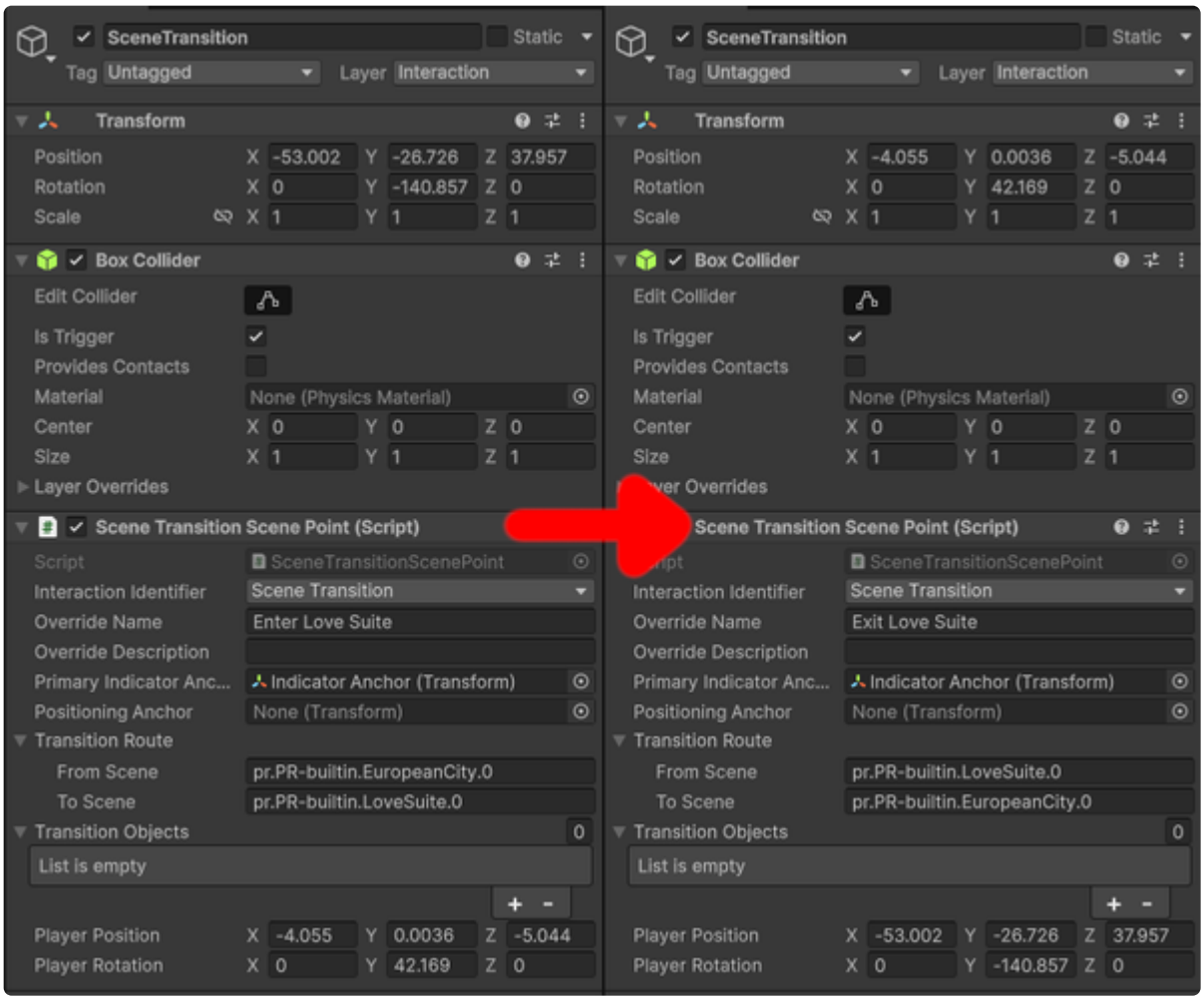
To set it up, right click the Hierarchy and create a **Scene Transition Scene Point** by using the *ScenePoint -> Scene Transition* menu item. Move the object as needed to where you want the interaction to be found.

If needed, a `Primary Indicator Anchor` can be specified to adjust the location of the primary interaction UI. An `Override Name` can also be specified to change the interactions name in the actionwheel and primary interaction UI.

Then, the `Transition Route` must be set up accordingly to transfer the data from the current scene (in this case it is European City) to the new scene (in this case, Love Suite). As valid route objects, the full scene GUIDs must be specified. As of right now `Transition Objects` are not used, so it should be left empty.

Lastly, the `Player Position` and `Player Rotation` vectors indicate where the player (and optionally partners) will end up placed in the **new scene**.

As a setup example, this is currently used to make the European City and Love Suite traversable by using a transition scene point in both scenes.

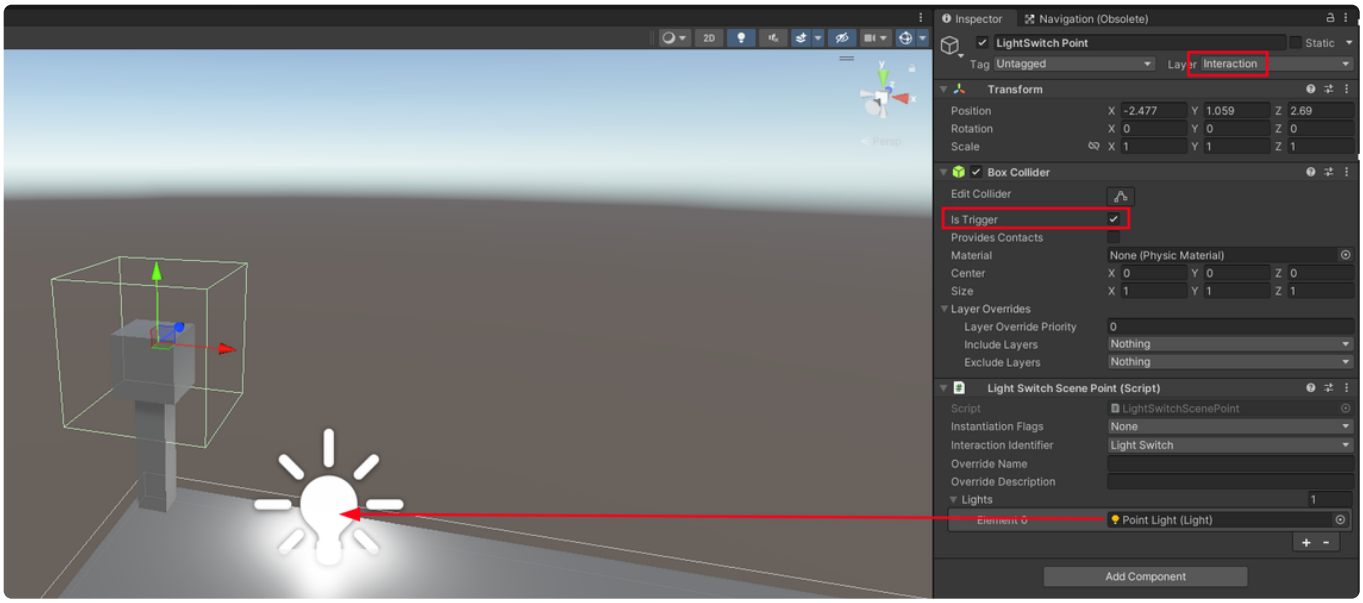

SceneTransition
Static
Tag Untagged
Layer Interaction
Transform
Position X -53.002 Y -26.726 Z 37.957
Rotation X 0 Y -140.857 Z 0
Scale X 1 Y 1 Z 1
Box Collider
Edit Collider
Is Trigger
Provides Contacts
Material None (Physics Material)
Center X 0 Y 0 Z 0
Size X 1 Y 1 Z 1
Layer Overrides
Scene Transition Scene Point (Script)
Script SceneTransitionScenePoint
Interaction Identifier Scene Transition
Override Name Enter Love Suite
Override Description
Primary Indicator Anc... Indicator Anchor (Transform)
Positioning Anchor None (Transform)
Transition Route
From Scene pr.PR-builtin.EuropeanCity.0
To Scene pr.PR-builtin.LoveSuite.0
Transition Objects 0
List is empty
Player Position X -4.055 Y 0.0036 Z -5.044
Player Rotation X 0 Y 42.169 Z 0
SceneTransition
Static
Tag Untagged
Layer Interaction
Transform
Position X -4.055 Y 0.0036 Z -5.044
Rotation X 0 Y 42.169 Z 0
Scale X 1 Y 1 Z 1
Box Collider
Edit Collider
Is Trigger
Provides Contacts
Material None (Physics Material)
Center X 0 Y 0 Z 0
Size X 1 Y 1 Z 1
Scene Transition Scene Point (Script)
SceneTransitionScenePoint
Interaction Identifier Scene Transition
Override Name Exit Love Suite
Override Description
Primary Indicator Anc... Indicator Anchor (Transform)
Positioning Anchor None (Transform)
Transition Route
From Scene pr.PR-builtin.LoveSuite.0
To Scene pr.PR-builtin.EuropeanCity.0
Transition Objects 0
List is empty
Player Position X -53.002 Y -26.726 Z 37.957
Player Rotation X 0 Y -140.857 Z 0

Light Switch

This interaction is used to toggle lights. To set it up, right click the Hierarchy and create a **Light Switch Scene Point** by using the *ScenePoint -> LightSwitch Point* menu item.

After calling the menu item, a new GameObject should be created with a collider and the lightswitch scene point. Make sure the objects layer is set to **Interaction**. Adjust the location of the object to be where you want your light switch to be. Keep the collider larger to make it easier to find the interaction.

Finally, assign the lights you want toggled to the **Lights** property of the lightswitch scenepoint.



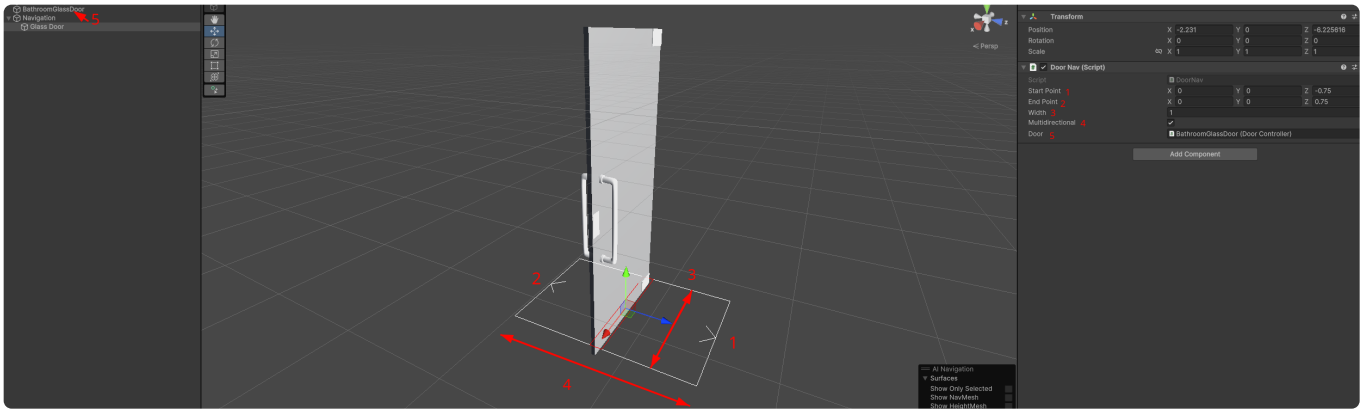
Door

This interaction is used to open and close doors. To set it up, please refer to the `Studio Components` documents **Door Controller** section. Make sure that the object which has the controller component has its layer set to **Interaction**.

Doors can be integrated into the navigation mesh so NPCs can open them and access areas closed off by doors. To do this, create a **separate** object from the door that does not move, and assign the **Door Nav** component to it.

Then, proceed with the following:

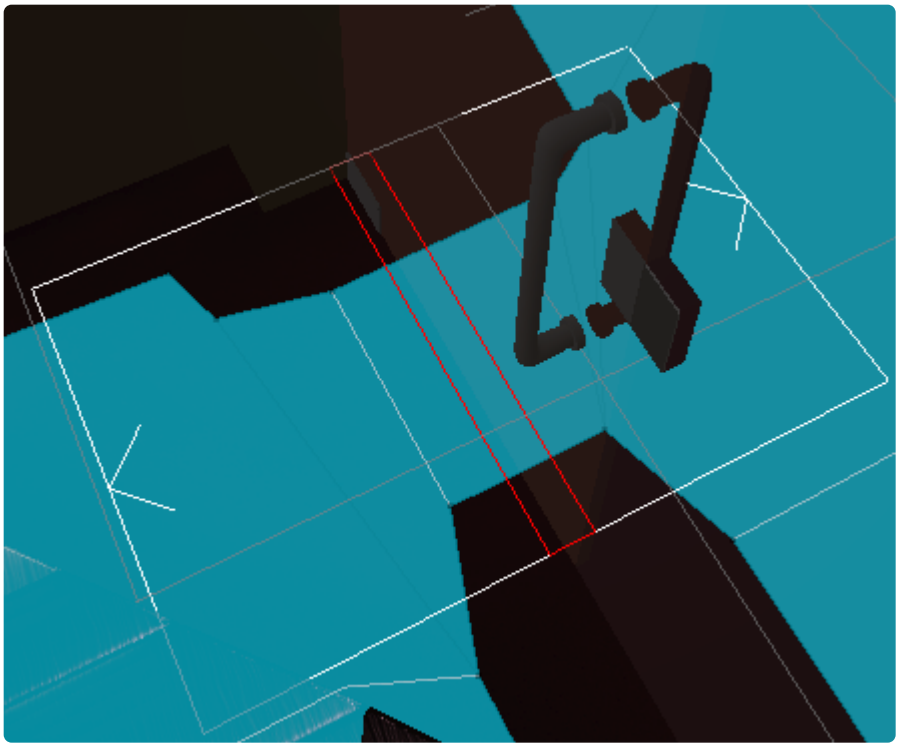
- Set up the **Start Point** (1) and **End Point** (2) to be the trigger points where the npc has to walk to to open the door.
- Specify a **Width** (3) that is around the width of the door. (In case of dual doors, make it wide enough to cover both doors)
- If set as **Multidirectional** (4), the NPC can open the door from either of the start/end points. Otherwise, only the start point is a possible location.
- Lastly, assign the **Door** (5) field to the **DoorController** component of the door itself.



As a small checklist, make sure that:

1. The door object is set to the **Interaction** layer.
2. The red box drawn by **DoorNav** is as close as possible to the center of the door and covers as much of it as possible.
3. The **NavMesh Surface** components Object Collection layers **exclude** the **Interaction** layer.

Once the navmesh is baked, the navmesh should appear to go through the door, as if it does not exist. If it is not connecting, see the checklist again.

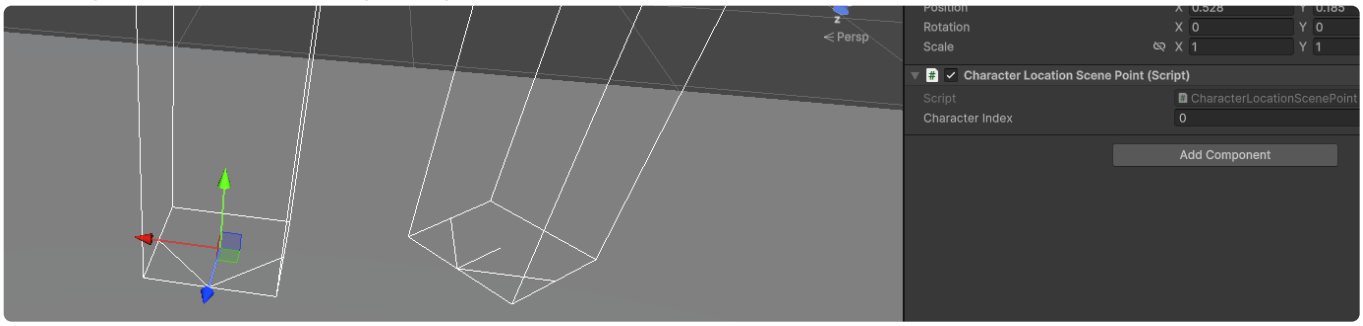


Character Locations

When starting H, characters are spawned at origin (0, 0, 0). This might not be acceptable with most scenes and a custom location should be used.

To set it up, right click the Hierarchy and create a **Character Location Scene Point** by using the *ScenePoint -> CharacterLocation Point* menu item. Move the object as needed to position where the character is spawned and what direction they will face.

Set the **Character Index** to either 0 (player) or 1 (partner). It is recommended to create two of these scene points, one for each participant.



H Locations

To allow H in various locations and play more animations, you should set up H locations for the custom scene.

There are currently 3 types of location:

- Flat
  - Standing or lying down on a flat surface.
- Seated
  - Sitting down on an edge of a surface like a bed.
- Wall
  - Leaning on or against a wall.

Each location type plays different animation types and requires different placement.

To create a H location, right click the Hierarchy and create a **HLocation** by using the *ScenePoint -> HLocation Point* menu item. Assign a **Name**, for example Table, Bed - Edge, Floor. Then, assign a **Type** for a custom gizmo to appear which should assist in placement of the location.

Make sure the object is set to the **HLocation** layer, collider is set to be a Trigger and is of an appropriate size.

